



ING 3 IA --- Mobile Applications Practicum

Practical 4: Data storage

1) **Introduction** : the objective of this practical is to learn how to ensure data persistence in a Flutter application. The "***Hive***" approach is used to locally store data representing the days of the week and their corresponding list of meals.

Additionally, this practical incorporate the use of "***Firestore_auth***" to implement user authentication through the "*login*" and "*signup*" pages.

2) **Local data storage with "*Hive*"** :

- a) Start by adding all the necessary dependencies using the command "*flutter pub add*" to import the packages: ***hive***, ***hive_flutter***, ***hive_generator*** and ***build_runner***.
- b) Add *Hive* annotations to the "*MealsOfADay*" and "*Meal*" data models, ensuring that each model has a unique "*typeId*". For example, the "*Meal*" data model can be annotated as follows:

```
import 'package:hive/hive.dart';
part 'meal.g.dart';

@HiveType(typeId: 0)
class Meal {
  @HiveField(0)
  final String name;
  @HiveField(1)
  final String imgPath;
  @HiveField(2)
  final List<String> listofIngredients;
  @HiveField(3)
  final String? identifier; }
```

- c) Generate a *Hive Model Class Adapter* for each data model

("MealsOfADay" and "Meal") by running the command: `"flutter packages pub run build_runner build"`. This command will generate the files `"meals_of_a_day.g.dart"` and `"meal.g.dart"`.

- d) From the `"Main()"` method, initialize Hive and register the generated adapters as shown below:

```
Future<void> main() async {  
  // Assurer l'initialisation de Flutter avant les opérations asynchrones  
  WidgetsFlutterBinding.ensureInitialized();  
  await Hive.initFlutter();  
  Hive.registerAdapter(MealAdapter());  
  Hive.registerAdapter(MealsOfADayAdapter());  
  ... }  
}
```

- e) Add data to the database:

1. Initially, in the `"HomeScreen"` class, a Hive database (Box) named `"MealsBDD"` of type `"MealsOfADay"` should be created. When the app is launched for the first time, this box should store the initial list of meals for each weekday. To implement this, the `"HomeScreen"` class must first be converted into a `Statefull` widget. Then, in the `initState()` method, check if the database is being created for the first time (i.e. it is empty). If so, initialize the box by adding all elements in the `"week_days_list"` using the `"put"` method. The `"key"` argument should correspond to the `"day"` attribute, while the `"value"` argument should be the element itself.

```
late Box <MealsOfADay> dayMealsBox;  
// Create the box (the database)  
dayMealsBox = await Hive.openBox<MealsOfADay>('MealsBDD');  
// Check if this is the first time and add initial meals for each week day.  
if (dayMealsBox.isEmpty) { for (var e in week_days_list) {  
  await dayMealsBox.put(e.day, e); }  
}
```

2. In the same way, data of type `"MealsOfADay"` can be added to the box through the `"add"` icon button in the `"WeekDaysCard"` widget. To do this, open the box and use the `"put"` method. The `"key"` argument should be set to the

"*day*" attribute of the "*dayAndItsListOfMeals*" variable, while the "*value*" argument corresponds to the "*dayAndItsListOfMeals*" variable. Be sure to include the new data in the "*listOfMealsForADay*" attribute of the "*dayAndItsListOfMeals*" variable.

```
final dayMealsBox = await Hive.openBox<MealsOfADay>('MealsBDD');
dayMealsBox.put(dayAndItsListOfMeals.day, dayAndItsListOfMeals);
```

- f) **Get data from the datadase:** to retrieve data from the box, use the "*get*" method via the "*visibility*" icon button of the "*WeekDaysCard*" widget. Then pass it as argument to the *PushNamed* method. Make sure to handle the exception when opening the box.

```
final dayMealsBox = await Hive.openBox<MealsOfADay>('MealsBDD');
Navigator.pushNamed(context, "Meals",
arguments: dayMealsBox.get(dayAndItsListOfMeals.day);
```

- g) **Remove data from the datadase:** when the "*Delete*" button is clicked, the selected meal should disappear from the "*MealsOfADayScreen*" interface. To achieve this:

1. Create a new method named "*deleteAMeal*" in the "*MealsOfADayScreen*" class. This method should take the meal to delete and the "*dayAndItsListOfMeals*" variable as parameters. Its purpose is to update the UI by removing the meal from the "*listOfMealsForADay*" attribute of "*dayAndItsListOfMeals*" and update the database by putting back "*dayAndItsListOfMeals*" into the box.

```
Future<void> deleteAMeal(Meal mealToRemove, dayAndItsListOfMeals ) async {
// Update the UI
setState(() {dayAndItsListOfMeals.listOfMealsForADay.remove(mealToRemove);}) ;
// Open the Hive box
Box<MealsOfADay> dayMealsBox = Hive.box<MealsOfADay>('MealsBDD');
// Update the box
await dayMealsBox.put(dayAndItsListOfMeals.day, dayAndItsListOfMeals);
}
```

2. Add a new attribute named *"onDeleteMeal"* of type *Function* to the *"MealCard"* class. This function should be triggered when the *"Delete"* button is pressed.
3. Adjust the *"MealCard"* widget returned from the *GridView.builder* in the *"MealsOfADayScreen"* class so that it passes the *"deleteAMeal"* method as the value for the *"onDeleteMeal"* function parameter. This ensures that when the *"onDeleteMeal"* function of the *"MealCard"* class is called, it executes the *"deleteAMeal"* method from the *"MealsOfADayScreen"* class.

```
return MealCard(  
  meal: dayAndItsListOfMeals.listOfMealsForADay[index],  
  onDeleteMeal: () => deleteAMeal(dayAndItsListOfMeals.listOfMealsForADay[index], dayAndItsListOfMeals ),  
);
```

- h) *Close the database:* from the *dispose()* method of the *"HomeScreen"* class, free up storage memory by closing the database.

```
@override  
void dispose() {  
  // Close the Hive box when the widget is deleted  
  dayMealsBox.close();  
  super.dispose();  
}
```

- 3) **User authentication with "Firebase auth"** : after the completion of all the **configuration steps of adding Firebase to your Flutter (ref. Course and tutoFirebaseSetup file)**, implement the following scenarios:

- a) The click on the *"signup"* button of the *"SignupScreen"* page leads to the creation of a user account with the email address and the password entered via the page form (do not forget to handle exceptions).

```
await FirebaseAuth.instance.createUserWithEmailAndPassword(  
    email: emailController.text,  
    password: pwdController.text,  
);
```

Subsequently, the existence of the email address must be verified to activate the created account. To do this, you need to use the ***sendEmailVerification*** method to send the activation link to the email address. Then, the user is directed to the "***LoginScreen***" page.

```
FirebaseAuth.instance.currentUser!.sendEmailVerification();
```

b) The click on the "***login***" button of the "***LoginScreen***" page allows the user to login and switch from the Sign-out state to the Sign-in state (do not forget to handle exceptions).

```
UserCredential userCredential = await  
FirebaseAuth.instance.signInWithEmailAndPassword (  
    email: emailController.text,  
    password: pwdController.text,  
);
```

Check if the user has already verified his email address. If verified, redirected him to the "***HomeScreen***" page. Otherwise, he will remain on the "***LoginScreen***" page.

```
if (userCredential.user!.emailVerified)
```

Optionally you can use a *SnackBar* to notify the user to check his email to activate his account.

c) The click on the "***Forgot your password ?***" text button of the "***LoginScreen***" page leads to the sending of a reset link to the provided email address.

```
await FirebaseAuth.instance.sendPasswordResetEmail( email: emailController.text);
```

Before sending the reset link, check if the email address is valid (using the *emailValidationFct* function) and verify that the user already exists (using the *fetchSignInMethodsForEmail* method, which returns the list of available sign-in methods for a given email).

```
List<String> signInMethods = await FirebaseAuth.instance
    .fetchSignInMethodsForEmail(emailController.text);
```

d) From the *AppBar action button* on the "**HomeScreen**", "**IngredientsScreen**" and "**MealsOfADayScreen**" pages, log the user out.

```
await FirebaseAuth.instance.signOut();
```

Use the *pushNamedAndRemoveUntill* method to display the "**LoginScreen**" page after logging out.